

```
In [ ]: import csv
import warnings
from collections import Counter
from pathlib import Path
from time import perf_counter

import pandas as pd
from ortools.sat.python import cp_model

pd.set_option("display.max_rows", 200) # mostrar os horários completos,
também na exportação para PDF

DIAS = ["Seg", "Ter", "Qua", "Qui", "Sex"]
PERIODOS = range(1, 6)

In [ ]: import marimo as mo
```

1. Descrição do problema e abordagem

Objetivo — construir um gerador de horários semanais a partir dos ficheiros CSV, respeitando as regras das turmas, disciplinas, professores e salas.

Regras principais

- Cada disciplina tem uma carga semanal e um professor associado.
- Algumas disciplinas precisam de uma sala especial ou de blocos de dois períodos consecutivos.
- Uma turma, um professor ou uma sala não podem ter aulas sobrepostas.
- As indisponibilidades dos professores e a capacidade das salas têm de ser respeitadas.

Estratégia de resolução

Usaremos programação por restrições com **CP-SAT**, do OR-Tools. Cada colocação possível de uma aula será representada por uma decisão booleana. O solver procurará primeiro uma solução que cumpra todas as regras obrigatórias e, depois, minimizará os tempos livres entre aulas de cada professor no mesmo dia (os «buracos»).

Os dados serão sempre lidos dos CSV, pelo que o modelo poderá ser usado com outros conjuntos de turmas, disciplinas, salas e indisponibilidades no mesmo

formato. No final, o notebook compara uma resolução independente com uma atualização baseada em H0. O uso do ChatGPT está identificado na secção «Uso de ferramentas LLM», no fim do notebook.

Modelo e validação

Os CSV são lidos com a biblioteca `csv` e preparados antes da modelação. O CP-SAT cria uma variável booleana para cada possível início de aula. Blocos duplos só podem começar nos períodos 1 a 4 e ocupam também o período seguinte.

O modelo aplica R1-R7: conflitos de turma, carga semanal, limite diário por disciplina, blocos duplos, conflitos e disponibilidade dos professores e capacidade de salas. R8 é garantida pela leitura dos quatro CSV. A função `validar_horario` verifica de forma independente as regras no horário extraído.

A atualização incremental dá ao solver H0 como pistas e maximiza as colocações de aula preservadas. Depois, um segundo modelo CP-SAT atribui as salas normais e maximiza quantas aulas mantêm a sala antiga, respeitando as sobreposições e as salas temporariamente indisponíveis. As alterações de horário e sala são contadas em conjunto por aula.

Justificação das escolhas

CP-SAT (OR-Tools). O problema é combinatório: cada decisão é booleana (a aula começa ou não num dado dia e período) e todas as regras R1-R7 se escrevem como somas dessas decisões limitadas por uma constante (carga exata, no máximo uma aula por período, capacidade de salas). O CP-SAT trata diretamente este tipo de restrições lineares sobre booleanos e, no mesmo modelo, otimiza um objetivo (os buracos de O1). Além disso:

- devolve o estado da resolução (`OPTIMAL` , `FEASIBLE` , `INFEASIBLE`), o que permite distinguir «não há horário» de «não houve tempo»;
- aceita um limite de tempo, como o enunciado admite para O1;
- aceita pistas (`add_hint`), que usamos para reaproveitar H0 em R9.

Para os resultados serem reproduzíveis, o solver corre com um só processo e semente fixa (`random_seed = 0`): os mesmos CSV produzem sempre o mesmo H0 e os mesmos H1. Só os tempos medidos variam.

Um solver SAT puro ou SMT obrigaria a codificar à mão as cardinalidades e o objetivo. Por isso seguimos a sugestão da disciplina.

Biblioteca `csv` para a leitura. Os ficheiros são pequenos e cada linha é validada individualmente antes da modelação (colunas obrigatórias, tipos, valores `sim / nao`, dias e períodos válidos, salas existentes). Com `csv.DictReader` cada linha chega como um dicionário de texto, que convertamos e verificamos explicitamente, com mensagens de erro claras. O modelo não usa operações tabulares (junções, agrupamentos), por isso o pandas não traria vantagem na leitura e acrescentaria conversões automáticas de tipos (por exemplo, `sala_especial` vazia passaria a `NaN`). Usamos o pandas apenas para apresentar o horário em tabela.

Para registar uma sala temporariamente indisponível, pode acrescentar-se a coluna opcional `sala` a `disponibilidade_excecoes.csv`; cada linha indica a sala, o dia e o período. Os CSV antigos sem essa coluna continuam válidos.

```
In [ ]: def ler_csv(caminho, colunas):
        """Lê um CSV (aceita BOM), confirma as colunas obrigatórias e devolve
        as linhas como dicionários."""
        with open(caminho, newline="", encoding="utf-8-sig") as ficheiro:
            leitor = csv.DictReader(ficheiro)
            em_falta = set(colunas) - set(leitor.fieldnames or [])
            if em_falta:
                raise ValueError(f"{caminho}: faltam as colunas {em_falta}")
            return list(leitor)
```

```
In [ ]: def carregar_dados(pasta):
        """Lê os quatro CSV da pasta e devolve turmas, disciplinas, salas e
        exceções."""
        # LLM (ChatGPT) – Registo 1 (passo 1): sugeriu reunir a leitura dos
        # quatro CSV numa função.
        pasta = Path(pasta)
        turmas = ler_csv(pasta / "turmas.csv", ["turma"])
        disciplinas = ler_csv(
            pasta / "disciplinas.csv",
            ["disciplina", "professor", "carga_semanal", "duplo_periodo",
            "sala_especial"],
        )
        salas = ler_csv(pasta / "salas.csv", ["sala", "tipo", "quantidade"])
        excecoes = ler_csv(pasta / "disponibilidade_excecoes.csv",
            ["professor", "dia", "periodo"])
        for excecao in excecoes: # coluna opcional `sala`: sala
            # temporariamente indisponível
            excecao.setdefault("sala", "")
        return turmas, disciplinas, salas, excecoes
```

```
In [ ]: def preparar_dados(dados):
        """Limpa e valida os dados lidos dos CSV, convertendo números e valores
        de duplo período."""
        # LLM (ChatGPT) – Registos 1 (passo 2) e 2: descreveu as conversões de
        # tipos e as validações a fazer.
        linhas_turmas, linhas_disciplinas, linhas_salas, linhas_excecoes =
        dados
```

```

turmas = [linha["turma"].strip() for linha in linhas_turmas]
if not turmas or not all(turmas):
    raise ValueError("Há turmas sem nome")
if len(turmas) != len(set(turmas)):
    raise ValueError("Há turmas repetidas")

salas = []
for linha in linhas_salas:
    sala = {
        "sala": linha["sala"].strip(),
        "tipo": linha["tipo"].strip().lower(),
        "quantidade": int(linha["quantidade"]),
    }
    if sala["tipo"] not in ("normal", "especial"):
        raise ValueError(f"Tipo de sala inválido: {sala['tipo']}")
    if sala["quantidade"] < 1:
        raise ValueError(f"Quantidade inválida para a sala {sala['sala']}")
    if not sala["sala"]:
        raise ValueError("Há salas sem nome")
    if any(sala["sala"] == outra["sala"] for outra in salas):
        raise ValueError(f"Sala repetida: {sala['sala']}")
    salas.append(sala)
salas_por_nome = {sala["sala"]: sala for sala in salas}

disciplinas = []
for linha in linhas_disciplinas:
    nome = linha["disciplina"].strip()
    professor = linha["professor"].strip()
    duplo = linha["duplo_periodo"].strip().lower()
    carga = int(linha["carga_semanal"])
    sala_especial = linha["sala_especial"].strip()

    if not nome:
        raise ValueError("Há disciplinas sem nome")
    if any(nome == d["disciplina"] for d in disciplinas):
        raise ValueError(f"Disciplina repetida: {nome}")
    if not professor:
        raise ValueError(f"Falta o professor da disciplina {nome}")
    if duplo not in ("sim", "nao"):
        raise ValueError(f"Valor inválido em duplo_periodo: {linha['duplo_periodo']}")
    if carga < 1 or (duplo == "sim" and carga % 2 != 0):
        raise ValueError(f"Carga semanal inválida para {nome}")
    if sala_especial and salas_por_nome.get(sala_especial, {}).get("tipo") != "especial":
        raise ValueError(f"Sala especial inexistente: {sala_especial}")

    disciplinas.append({
        "disciplina": nome,
        "professor": professor,
        "carga_semanal": carga,
        "duplo_periodo": duplo == "sim",
        "sala_especial": sala_especial or None,
    })

```

```

professores = {d["professor"] for d in disciplinas}
excecoes = []
for linha in linhas_excecoes:
    dia = linha["dia"].strip()
    periodo = int(linha["periodo"])
    if dia not in DIAS or periodo not in PERIODOS:
        raise ValueError(f"Dia ou período inválido: {dia}, {periodo}")

    sala = linha.get("sala", "").strip()
    if sala:
        if sala not in salas_por_nome:
            raise ValueError(f"Sala desconhecida na exceção: {sala}")
        excecoes.append({"sala": sala, "dia": dia, "periodo": periodo})
        continue

    professor = linha["professor"].strip()
    if professor not in professores: # pode ser uma exceção antiga de
        um professor substituído
        warnings.warn(
            f"Exceção ignorada para professor desconhecido ou
substituído: {professor}",
            UserWarning,
            stacklevel=2,
        )
        continue
    excecoes.append({"professor": professor, "dia": dia, "periodo":
periodo})

    bloqueadas = Counter((e["sala"], e["dia"], e["periodo"]) for e in
excecoes if "sala" in e)
    for (nome, dia, periodo), quantidade in bloqueadas.items():
        if quantidade > salas_por_nome[nome]["quantidade"]:
            raise ValueError(f"Demasiadas salas {nome} indisponíveis em
{dia}, período {periodo}")

    return turmas, disciplinas, salas, excecoes

```

```

In [ ]: def duracao(disciplina):
        """Número de períodos que cada aula da disciplina ocupa (2 se for bloco
duplo)."""
        return 2 if disciplina["duplo_periodo"] else 1

```

```

In [ ]: def criar_modelo(dados):
        """Cria o modelo CP-SAT com uma variável booleana por início possível
de aula, indexada por (turma, disciplina, dia, período)."""
        # LLM (ChatGPT) – Registo 4: propôs uma variável booleana por colocação
possível, com blocos duplos só a começar nos períodos 1 a 4.
        turmas, disciplinas, _, _ = dados
        modelo = cp_model.CpModel()
        x = {}
        for turma in turmas:
            for disciplina in disciplinas:
                for dia in DIAS:
                    for periodo in PERIODOS:

```

```

        if disciplina["duplo_perodo"] and periodo == 5:
            continue # um bloco duplo não pode começar no
último período

        chave = (turma, disciplina["disciplina"], dia, periodo)
        x[chave] = modelo.new_bool_var(f"x_{len(x)}")

    return modelo, x

```

```

In [ ]: def ocupantes(x, turma, disciplina, dia, periodo):
    """Variáveis da disciplina que ocupam um período: o início nele ou, nos
    blocos duplos, no período anterior."""
    # LLM (ChatGPT) – Registo 6: indicou que uma aula dupla ocupa o período
    de início e o seguinte.
    inicios = [periodo, periodo - 1] if disciplina["duplo_perodo"] else
[periodo]
    chaves = [(turma, disciplina["disciplina"], dia, i) for i in inicios]
    return [x[chave] for chave in chaves if chave in x]

```

```

In [ ]: def adicionar_restricoes(modelo, x, dados):
    """Acrescenta ao modelo as regras R1–R7: carga, limite diário,
    conflitos, disponibilidade e salas."""
    # LLM (ChatGPT) – Registos 5 e 6: descreveu como escrever R1–R7 como
    somas de variáveis; as quatro funções sugeridas foram juntadas numa só.
    turmas, disciplinas, salas, excecoes = dados
    prof_indisponivel = {(e["professor"], e["dia"], e["periodo"]) for e in
excecoes if "professor" in e}
    salas_bloqueadas = Counter((e["sala"], e["dia"], e["periodo"]) for e in
excecoes if "sala" in e)

    for turma in turmas:
        for d in disciplinas:
            nome = d["disciplina"]
            # R2: carga semanal exata (em blocos, se a disciplina for
dupla).
            todas = [x[(turma, nome, dia, p)] for dia in DIAS for p in
PERIODOS if (turma, nome, dia, p) in x]
            modelo.add(sum(todas) == d["carga_semanal"] // duracao(d))
            for dia in DIAS:
                # R3: no máximo uma ocorrência da disciplina por dia.
                do_dia = [x[(turma, nome, dia, p)] for p in PERIODOS if
(turma, nome, dia, p) in x]
                modelo.add(sum(do_dia) <= 1)
                # R6: nenhum período da aula pode ser de indisponibilidade
do professor.
                for p in PERIODOS:
                    if (turma, nome, dia, p) in x and any(
                        (d["professor"], dia, p + i) in prof_indisponivel
for i in range(duracao(d))
                    ):
                        modelo.add(x[(turma, nome, dia, p)] == 0)

    professores = {d["professor"] for d in disciplinas}
    for dia in DIAS:
        for p in PERIODOS:
            # R1: uma turma só tem uma aula de cada vez.
            for turma in turmas:

```



```

        modelo.add(buraco >= tem[antes] + tem[depois] -
tem[p] - 1)
    modelo.minimize(sum(buracos))

```

```

In [ ]: def nomes_salas_normais(salas):
    """Nomes individuais das salas normais por tipo, p. ex. {'Sala Normal':
['Sala Normal 1', ...]}."""
    return {
        s["sala"]: [s["sala"]] if s["quantidade"] == 1 else [f"{s['sala']}"
{n}]" for n in range(1, s["quantidade"] + 1)]
        for s in salas
        if s["tipo"] == "normal"
    }

```

```

In [ ]: def salas_bloqueadas(salas, excecoes):
    """Mapa (dia, período) -> nomes das salas normais indisponíveis (as
últimas N de cada tipo)."""
    por_tipo = nomes_salas_normais(salas)
    contagem = Counter((e["sala"], e["dia"], e["período"]) for e in
excecoes or [] if "sala" in e)
    bloqueadas = {}
    for (tipo, dia, período), n in contagem.items():
        bloqueadas.setdefault((dia, período),
set()).update(por_tipo.get(tipo, [])[-n:])
    return bloqueadas

```

```

In [ ]: def atribuir_salas_normais(horario, salas, horario_antigo=None,
excecoes_salas=None, limite_segundos=30):
    """Atribui salas físicas às aulas normais (sem conflitos) e prefere
manter as salas do horário anterior."""
    nomes = [n for ns in nomes_salas_normais(salas).values() for n in ns]
    bloqueadas = salas_bloqueadas(salas, excecoes_salas)
    antigas = {(a["turma"], a["disciplina"], a["dia"], a["período"]):
a["sala"] for a in horario_antigo or []}
    aulas = [(i, a) for i, a in enumerate(horario) if a["sala"] is None]
    if not aulas:
        return horario

    modelo = cp_model.CpModel()
    var = {}
    mantidas = []
    for i, a in aulas:
        periodos = range(a["período"], a["período"] + a["duracao"])
        for sala in nomes:
            if any(sala in bloqueadas.get((a["dia"], p), ()) for p in
periodos):
                continue
            var[(i, sala)] = modelo.new_bool_var(f"sala_{len(var)}")
            if antigas.get((a["turma"], a["disciplina"], a["dia"],
a["período"])) == sala:
                mantidas.append(var[(i, sala)])
            modelo.add(sum(var[(i, s)] for s in nomes if (i, s) in var) == 1)

    for sala in nomes: # uma sala não pode ter duas aulas ao mesmo tempo
        for dia in DIAS:

```

```

        for p in PERIODOS:
            modelo.add(sum(
                var[(i, sala)] for i, a in aulas
                if a["dia"] == dia and a["periodo"] <= p < a["periodo"]
+ a["duracao"] and (i, sala) in var
                ) <= 1)

    modelo.maximize(sum(mantidas))
    solver = cp_model.CpSolver()
    solver.parameters.num_search_workers = 1
    solver.parameters.random_seed = 0
    if limite_segundos is not None:
        solver.parameters.max_time_in_seconds = limite_segundos
    estado = solver.solve(modelo)
    if estado == cp_model.INFEASIBLE:
        raise ValueError("Não há salas normais disponíveis para o horário")
    if estado not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
        raise TimeoutError("O solver não conseguiu atribuir as salas a
tempo")

    for (i, sala), variavel in var.items():
        if solver.value(variavel):
            horario[i]["sala"] = sala
    return horario

```

```

In [ ]: def construir_horario(solver, x, dados):
    """Transforma as decisões verdadeiras do solver em aulas; as salas
    normais ficam por atribuir (None)."""
    # LLM (ChatGPT) – Registo 8 (passo 2): corresponde a `extrair_horario`,
    sugerida pelo ChatGPT.
    por_nome = {d["disciplina"]: d for d in dados[1]}
    ocorrencias = Counter()
    horario = []
    for (turma, nome, dia, inicio), variavel in x.items():
        if solver.value(variavel):
            ocorrencias[(turma, nome)] += 1
            d = por_nome[nome]
            horario.append({
                "id": (turma, nome, ocorrencias[(turma, nome)]),
                "turma": turma,
                "disciplina": nome,
                "professor": d["professor"],
                "dia": dia,
                "periodo": inicio,
                "duracao": duracao(d),
                "sala": d["sala_especial"],
            })
    return horario

```

```

In [ ]: def resolver_com_salas(modelo, x, dados, limite_segundos=30,
    horario_antigo=None):
    """Resolve o horário; se não houver salas físicas para ele, exclui-o do
    modelo e pede outro."""
    # LLM (ChatGPT) – Registo 8 (passo 1): corresponde a `resolver_modelo`,
    sugerida pelo ChatGPT; a atribuição de salas foi acrescentada depois.

```

```

    fim = None if limite_segundos is None else perf_counter() +
limite_segundos
    while True:
        solver = cp_model.CpSolver()
        # Um só processo e semente fixa: a mesma entrada dá sempre o mesmo
        horário (resultados reproduzíveis).
        solver.parameters.num_search_workers = 1
        solver.parameters.random_seed = 0
        restante = None if fim is None else fim - perf_counter()
        if restante is not None:
            if restante <= 0:
                return [], "UNKNOWN"
            solver.parameters.max_time_in_seconds = restante
            estado = solver.solve(modelo)
            if estado not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
                return [], solver.status_name(estado)

            restante = None if fim is None else fim - perf_counter()
            if restante is not None and restante <= 0:
                return [], "UNKNOWN"
            horario = construir_horario(solver, x, dados)
            try:
                horario = atribuir_salas_normais(horario, dados[2],
horario_antigo, dados[3], restante)
                return horario, solver.status_name(estado)
            except TimeoutError:
                return [], "UNKNOWN"
            except ValueError:
                escolhidas = [v for v in x.values() if solver.value(v)]
                modelo.add(sum(escolhidas) <= len(escolhidas) - 1)

```

```

In [ ]: def gerar_horario(dados_csv, limite_segundos=30):
    """Gera o horário do zero, otimizando os buracos; devolve o horário, o
    estado e os erros de validação."""
    # LLM (ChatGPT) – Registo 9: fluxo completo (dados → H0) sugerido pelo
    ChatGPT.
    dados = preparar_dados(dados_csv)
    modelo, x = criar_modelo(dados)
    adicionar_restricoes(modelo, x, dados)
    adicionar_objetivo_buracos(modelo, x, dados)
    horario, estado = resolver_com_salas(modelo, x, dados, limite_segundos)
    if estado in ("OPTIMAL", "FEASIBLE"):
        return horario, estado, validar_horario(horario, dados)
    return horario, estado, ["0 solver não encontrou um horário válido."]

```

```

In [ ]: def atualizar_horario(horario_antigo, dados_novos, limite_segundos=30,
    usar_pistas=True):
    """Gera um novo horário com os dados atualizados, preservando o máximo
    de aulas e salas de H0."""
    # LLM (ChatGPT) – Registos 1 (passo 9) e 8 (passo 6): sugeriu
    reaproveitar H0 e preservar as aulas que continuam válidas.
    dados = preparar_dados(dados_novos)
    modelo, x = criar_modelo(dados)
    adicionar_restricoes(modelo, x, dados)

```

```

    antigas = {(a["turma"], a["disciplina"], a["dia"], a["periodo"]) for a
in horario_antigo}
    if usar_pistas: # H0 como ponto de partida do solver
        for chave, variavel in x.items():
            modelo.add_hint(variavel, int(chave in antigas))
        modelo.maximize(sum(v for chave, v in x.items() if chave in antigas))

    horario_novo, estado = resolver_com_salas(modelo, x, dados,
limite_segundos, horario_antigo)
    if estado in ("OPTIMAL", "FEASIBLE"):
        erros = validar_horario(horario_novo, dados)
        if erros:
            raise ValueError("O novo horário falhou na validação:\n" +
"\n".join(erros))
        return horario_novo, estado, contar_alteracoes(horario_antigo,
horario_novo)

```

```

In [ ]: def validar_horario(horario, dados):
    """Verifica R1-R7 de forma independente do solver e devolve a lista de
    erros (vazia se o horário é válido)."""
    # LLM (ChatGPT) – Registos 1 (passo 8) e 8 (passo 4): sugeriu um
    validador independente do solver que devolve a lista de erros.
    turmas, disciplinas, salas, excecoes = dados
    por_nome = {d["disciplina"]: d for d in disciplinas}
    nomes_normais = nomes_salas_normais(salas)
    normais = [n for ns in nomes_normais.values() for n in ns]
    bloqueadas = salas_bloqueadas(salas, excecoes)
    prof_indisponivel = {(e["professor"], e["dia"], e["periodo"]) for e in
excecoes if "professor" in e}
    salas_indisponiveis = Counter((e["sala"], e["dia"], e["periodo"]) for e
in excecoes if "sala" in e)
    capacidade_especial = {s["sala"]: s["quantidade"] for s in salas if
s["tipo"] == "especial"}

    erros = []
    cargas, por_dia = Counter(), Counter()
    turma_ocupada, prof_ocupado, sala_ocupada, normais_ocupadas =
Counter(), Counter(), Counter(), Counter()

    for a in horario:
        turma, nome, dia, inicio = a["turma"], a["disciplina"], a["dia"],
a["periodo"]
        if turma not in turmas:
            erros.append(f"Turma desconhecida: {turma}")
            continue
        if nome not in por_nome:
            erros.append(f"Disciplina desconhecida: {nome}")
            continue
        d = por_nome[nome]
        if a["duracao"] != duracao(d):
            erros.append(f"Duração incorreta para {turma} - {nome}")
        if dia not in DIAS or inicio not in PERIODOS:
            erros.append(f"Dia ou período inválido para {turma} - {nome}")
            continue
        if d["duplo_periodo"] and inicio == 5:
            erros.append(f"Aula dupla começa no último período: {turma} -

```

```

{nome}")

        continue

    cargas[(turma, nome)] += duracao(d)
    por_dia[(turma, nome, dia)] += 1
    especial = d["sala_especial"]
    if especial and a["sala"] != especial:
        erros.append(f"Sala incompatível para {turma} - {nome}")
    if not especial and a["sala"] not in normais:
        erros.append(f"Sala normal incompatível para {turma} - {nome}")

    for p in range(inicio, inicio + duracao(d)):
        turma_ocupada[(turma, dia, p)] += 1
        prof_ocupado[(d["professor"], dia, p)] += 1
        sala_ocupada[(bool(especial), especial or a["sala"], dia, p)]
+= 1

        if (d["professor"], dia, p) in prof_indisponivel:
            erros.append(f"{d['professor']} indisponível em {dia},
período {p}")
        if not especial:
            normais_ocupadas[(dia, p)] += 1
            if a["sala"] in bloqueadas.get((dia, p), ()):
                erros.append(f"Sala {a['sala']} indisponível em {dia},
período {p}")

    for turma in turmas:
        for d in disciplinas:
            carga = cargas[(turma, d["disciplina"])]
            if carga != d["carga_semanal"]:
                erros.append(
                    f"Carga semanal incorreta para {turma} -
{d['disciplina']}: "
                    f"{carga} em vez de {d['carga_semanal']}"
                )
            erros += [f"Mais de uma ocorrência de {n} para {t} à {dia}" for (t, n,
dia), q in por_dia.items() if q > 1]
            erros += [f"Sobreposição na turma {t}, {dia}, período {p}" for (t, dia,
p), q in turma_ocupada.items() if q > 1]
            erros += [f"Sobreposição do professor {pr}, {dia}, período {p}" for
(pr, dia, p), q in prof_ocupado.items() if q > 1]
            for (especial, sala, dia, p), q in sala_ocupada.items():
                capacidade = max(0, capacidade_especial.get(sala, 0) -
salas_indisponiveis[(sala, dia, p)]) if especial else 1
                if q > capacidade:
                    erros.append(f"Capacidade excedida na sala {sala}, {dia},
período {p}")
            total_normais = sum(s["quantidade"] for s in salas if s["tipo"] ==
"normal")
            for (dia, p), q in normais_ocupadas.items():
                indisponiveis = sum(salas_indisponiveis[(tipo, dia, p)] for tipo in
nomes_normais)
                if q > max(0, total_normais - indisponiveis):
                    erros.append(f"Capacidade de salas normais excedida em {dia},
período {p}")
    return erros

```

```
In [ ]: def contar_alteracoes(horario_antigo, horario_novo):
        """Conta as aulas que mudaram de dia, período ou sala, incluindo aulas
        novas ou removidas."""
        # LLM (ChatGPT) – Registos 1 (passo 9) e 8 (passo 5): sugeriu
        # emparelhar as ocorrências da mesma turma e disciplina.
        def agrupar(horario):
            grupos = {}
            for a in horario:
                grupos.setdefault((a["turma"], a["disciplina"]), Counter())
            [(a["dia"], a["periodo"], a["sala"])] += 1
            return grupos

        antigos, novos = agrupar(horario_antigo), agrupar(horario_novo)
        total = 0
        for chave in set(antigos) | set(novos):
            a, n = antigos.get(chave, Counter()), novos.get(chave, Counter())
            total += max(sum(a.values()), sum(n.values())) - sum((a &
n).values())
        return total
```

```
In [ ]: def contar_buracos_horario(horario):
        """Conta os períodos livres entre a primeira e a última aula de cada
        professor em cada dia."""
        ocupacao = {}
        for a in horario:
            ocupacao.setdefault((a["professor"], a["dia"]),
set()).update(range(a["periodo"], a["periodo"] + a["duracao"]))
        return sum(max(ps) - min(ps) + 1 - len(ps) for ps in ocupacao.values())
```

```
In [ ]: def apresentar_horario(horario):
        """Organiza o horário numa tabela ordenada por turma, dia e período
        para mostrar no Marimo."""
        # LLM (ChatGPT) – Registos 1 (passo 7) e 8 (passo 3): sugeriu uma
        # tabela ordenada por turma, dia e período.
        colunas = ["turma", "dia", "periodo", "duracao", "disciplina",
"professor", "sala"]
        if not horario:
            return pd.DataFrame(columns=colunas)
        tabela = pd.DataFrame(horario)
        tabela["ordem_dia"] = tabela["dia"].map(DIAS.index)
        return tabela.sort_values(["turma", "ordem_dia", "periodo"])
[colunas].reset_index(drop=True)
```

```
In [ ]: def comparar_metodos(horario_h0, dados_csv, limite_segundos=30):
        """Corre a atualização incremental e a resolução do zero e devolve as
        linhas da tabela e os dois horários."""
        # LLM (ChatGPT) – Registos 1 (passo 9) e 9: sugeriu comparar com uma
        # resolução do zero, medindo o tempo e as aulas alteradas.
        dados = preparar_dados(dados_csv)
        linhas, horarios = [], []
        for metodo in ("Incremental com pistas de H0", "Resolução independente
do zero"):
            inicio = perf_counter()
            if metodo.startswith("Incremental"):
```

```

        horario, estado, _ = atualizar_horario(horario_h0, dados_csv,
limite_segundos)
    else:
        horario, estado, _ = gerar_horario(dados_csv, limite_segundos)
    linhas.append({
        "metodo": metodo,
        "estado": estado,
        "tempo": perf_counter() - inicio,
        "alteracoes": contar_alteracoes(horario_h0, horario),
        "buracos": contar_buracos_horario(horario),
        "erros": len(validar_horario(horario, dados)),
    })
    horarios.append(horario)
return linhas, horarios

```

```

In [ ]: def tabela_markdown(linhas):
    """Formata as linhas de comparação como tabela em markdown."""
    topo = "| Método | Estado | Tempo (s) | Aulas alteradas | Buracos | Erros |\n| --- | --- | ---: | ---: | ---: | ---: |"
    corpo = [
        f"| {l['metodo']} | {l['estado']} | {l['tempo']:.3f} | {l['alteracoes']} | {l['buracos']} | {l['erros']} |"
        for l in linhas
    ]
    return "\n".join([topo] + corpo)

```

```

In [ ]: def forcar_reparacao(horario_h0, dados_csv, professor="Prof. Ana"):
    """Copia os dados e bloqueia (só em memória) um período de uma aula do professor em H0; devolve os dados, a aula e o período."""
    ja_indisponivel = {(e["dia"], e["periodo"])} for e in preparar_dados(dados_csv)[3] if e.get("professor") == professor
    for aula in horario_h0:
        if aula["professor"] != professor:
            continue
        for p in range(aula["periodo"], aula["periodo"] + aula["duracao"]):
            if (aula["dia"], p) not in ja_indisponivel:
                novos = [[linha.copy() for linha in grupo] for grupo in dados_csv]
                novos[3].append({"professor": professor, "dia": aula["dia"], "periodo": str(p), "sala": ""})
                return novos, aula, p
        raise ValueError(f"Não foi encontrada uma aula disponível de {professor} em H0")

```

```

In [ ]: dados_csv = carregar_dados(Path(__file__).resolve().parent / "dados")
horario, estado, erros = gerar_horario(dados_csv)

resultado = mo.vstack([
    mo.md(f"***Estado do solver:** {estado}"),
    mo.md(f"***Erros de validação:** " + "; ".join(erros) if erros else "***Validação:** sem erros"),
    apresentar_horario(horario),
])

resultado

```

Estado do solver: OPTIMAL

Validação: sem erros

	turma	dia	periodo	duracao	disciplina	professor	sala
0	7ºA	Seg	1	1	Português	Prof. Bruno	Sala Normal 1
1	7ºA	Seg	2	1	Ciências	Prof. Carla	Laboratório
2	7ºA	Seg	3	1	Matemática	Prof. Ana	Sala Normal 1
3	7ºA	Seg	4	2	Educação Física	Prof. Eduardo	Ginásio
4	7ºA	Ter	1	1	Português	Prof. Bruno	Sala Normal 1
5	7ºA	Ter	2	1	Ciências	Prof. Carla	Laboratório
6	7ºA	Ter	3	1	Matemática	Prof. Ana	Sala Normal 1
7	7ºA	Ter	4	1	Inglês	Prof. Diana	Sala Normal 1
8	7ºA	Qua	3	1	Português	Prof. Bruno	Sala Normal 1
9	7ºA	Qua	4	1	Ciências	Prof. Carla	Laboratório
10	7ºA	Qua	5	1	Matemática	Prof. Ana	Sala Normal 1
11	7ºA	Qui	3	1	Inglês	Prof. Diana	Sala Normal 1
12	7ºA	Qui	4	1	História	Prof. Diana	Sala Normal 2
13	7ºA	Qui	5	1	Português	Prof. Bruno	Sala Normal 1
14	7ºA	Sex	3	1	Matemática	Prof. Ana	Sala Normal 1
15	7ºA	Sex	4	1	História	Prof. Diana	Sala Normal 2
16	7ºB	Seg	3	1	Inglês	Prof. Diana	Sala Normal 6
17	7ºB	Seg	4	1	Matemática	Prof. Ana	Sala Normal 1
18	7ºB	Ter	1	1	Ciências	Prof. Carla	Laboratório
19	7ºB	Ter	2	1	Português	Prof. Bruno	Sala Normal 1
20	7ºB	Ter	3	1	História	Prof. Diana	Sala Normal 6
21	7ºB	Ter	4	2	Educação Física	Prof. Eduardo	Ginásio
22	7ºB	Qua	2	1	Português	Prof. Bruno	Sala Normal 1
23	7ºB	Qua	4	1	Matemática	Prof. Ana	Sala Normal 1
24	7ºB	Qui	1	1	Matemática	Prof. Ana	Sala Normal 1
25	7ºB	Qui	2	1	História	Prof. Diana	Sala Normal 1
26	7ºB	Qui	4	1	Português	Prof. Bruno	Sala Normal 1
27	7ºB	Qui	5	1	Ciências	Prof. Carla	Laboratório
28	7ºB	Sex	1	1	Português	Prof. Bruno	Sala Normal 1
29	7ºB	Sex	2	1	Ciências	Prof. Carla	Laboratório
30	7ºB	Sex	3	1	Inglês	Prof. Diana	Sala Normal 6

	turma	dia	periodo	duracao	disciplina	professor	sala
31	7ºB	Sex	4	1	Matemática	Prof. Ana	Sala Normal 1

```
In [ ]: dados_v2 = carregar_dados(Path(__file__).resolve().parent / "dados_v2")
linhas_v2, (h1_incremental, h1_raiz) = comparar_metodos(horario, dados_v2)

dados_forcados, aula_afetada, periodo_afetado = forcar_reparacao(horario,
dados_v2)
linhas_forçadas, (h1_forcado_inc, h1_forcado_raiz) =
comparar_metodos(horario, dados_forcados)
ocupado = any(
    a["professor"] == aula_afetada["professor"]
    and a["dia"] == aula_afetada["dia"]
    and a["periodo"] <= periodo_afetado < a["periodo"] + a["duracao"]
    for a in h1_forcado_inc
)
reparacao_confirmada = (
    linhas_forçadas[0]["estado"] in ("OPTIMAL", "FEASIBLE")
    and linhas_forçadas[0]["erros"] == 0
    and not ocupado
)

comparacao = mo.vstack([
    mo.md(
        """## Comparação: H0 → dados_v2/\n\n
        "A primeira tabela mostra a alteração fornecida em `dados_v2/`. "
        "Cada método corre uma vez; os tempos variam e os objetivos "
        "são diferentes. A resolução independente minimiza os buracos; "
        "a incremental tenta manter as aulas de H0."
        ),
    mo.md("### Alteração fornecida em dados_v2/"),
    mo.md(tabela_markdown(linhas_v2)),
    mo.md(
        """### Reparação forçada de uma aula de H0\n\n
        f"Foi escolhida a aula {aula_afetada['disciplina']} da turma "
        f"{aula_afetada['turma']} em {aula_afetada['dia']}, período "
        f"{periodo_afetado}. Esse período foi acrescentado às "
        "indisponibilidades da Prof. Ana apenas nesta execução; não foi "
        "criado outro CSV.\n\n
        "A atualização incremental libertou o período: "
        f"{'sim' if reparacao_confirmada else 'não foi possível "
        confirmar'}."
        ),
    mo.md(tabela_markdown(linhas_forçadas)),
    mo.md("### H1 incremental com reparação forçada"),
    apresentar_horario(h1_forcado_inc),
    mo.md("### H1 resolvido do zero com reparação forçada"),
    apresentar_horario(h1_forcado_raiz),
    mo.md("### H1 incremental"),
    apresentar_horario(h1_incremental),
    mo.md("### H1 resolvido do zero"),
    apresentar_horario(h1_raiz),
])
```

comparacao

Comparação: H0 → dados_v2/

A primeira tabela mostra a alteração fornecida em dados_v2/. Cada método corre uma vez; os tempos variam e os objetivos são diferentes. A resolução independente minimiza os buracos; a incremental tenta manter as aulas de H0.

Alteração fornecida em dados_v2/

	Método	Estado	Tempo (s)	Aulas alteradas	Buracos	Erros
	Incremental com pistas de H0	OPTIMAL	0.022	2	0	0
	Resolução independente do zero	OPTIMAL	0.045	29	0	0

Reparação forçada de uma aula de H0

Foi escolhida a aula Matemática da turma 7ªA em Seg, período 3. Esse período foi acrescentado às indisponibilidades da Prof. Ana apenas nesta execução; não foi criado outro CSV. A atualização incremental libertou o período: sim.

	Método	Estado	Tempo (s)	Aulas alteradas	Buracos	Erros
	Incremental com pistas de H0	OPTIMAL	0.027	3	0	0
	Resolução independente do zero	OPTIMAL	0.046	26	0	0

H1 incremental com reparação forçada

	turma	dia	periodo	duracao	disciplina	professor	sala
0	7ªA	Seg	1	1	Português	Prof. Bruno	Sala Normal 1
1	7ªA	Seg	2	1	Ciências	Prof. Carla	Laboratório
2	7ªA	Seg	4	2	Educação Física	Prof. Eduardo	Ginásio
3	7ªA	Ter	1	1	Português	Prof. Bruno	Sala Normal 1
4	7ªA	Ter	2	1	Ciências	Prof. Carla	Laboratório
5	7ªA	Ter	3	1	Matemática	Prof. Ana	Sala Normal 1
6	7ªA	Ter	4	1	Inglês	Prof. Diana	Sala Normal 1
7	7ªA	Qua	3	1	Português	Prof. Bruno	Sala Normal 1
8	7ªA	Qua	4	1	Ciências	Prof. Carla	Laboratório

	turma	dia	periodo	duracao	disciplina	professor	sala
9	7ºA	Qua	5	1	Matemática	Prof. Ana	Sala Normal 1
10	7ºA	Qui	2	1	Matemática	Prof. Ana	Sala Normal 2
11	7ºA	Qui	3	1	Inglês	Prof. Diana	Sala Normal 1
12	7ºA	Qui	4	1	História	Prof. Diana	Sala Normal 2
13	7ºA	Qui	5	1	Português	Prof. Bruno	Sala Normal 1
14	7ºA	Sex	3	1	Matemática	Prof. Ana	Sala Normal 1
15	7ºA	Sex	4	1	História	Prof. Diana	Sala Normal 2
16	7ºB	Seg	3	1	Inglês	Prof. Diana	Sala Normal 6
17	7ºB	Seg	4	1	Matemática	Prof. Ana	Sala Normal 1
18	7ºB	Ter	1	1	Ciências	Prof. Carla	Laboratório
19	7ºB	Ter	2	1	Português	Prof. Bruno	Sala Normal 1
20	7ºB	Ter	3	1	História	Prof. Diana	Sala Normal 6
21	7ºB	Ter	4	1	Matemática	Prof. Ana	Sala Normal 5
22	7ºB	Qua	2	1	Português	Prof. Bruno	Sala Normal 1
23	7ºB	Qua	4	1	Matemática	Prof. Ana	Sala Normal 1
24	7ºB	Qui	1	1	Matemática	Prof. Ana	Sala Normal 1
25	7ºB	Qui	2	1	História	Prof. Diana	Sala Normal 1
26	7ºB	Qui	4	1	Português	Prof. Bruno	Sala Normal 1
27	7ºB	Qui	5	1	Ciências	Prof. Carla	Laboratório
28	7ºB	Sex	1	1	Português	Prof. Bruno	Sala Normal 1
29	7ºB	Sex	2	1	Ciências	Prof. Carla	Laboratório
30	7ºB	Sex	3	1	Inglês	Prof. Diana	Sala Normal 6
31	7ºB	Sex	4	2	Educação Física	Prof. Eduardo	Ginásio

H1 resolvido do zero com reparação forçada

	turma	dia	periodo	duracao	disciplina	professor	sala
0	7ºA	Seg	2	1	História	Prof. Diana	Sala Normal 2
1	7ºA	Seg	3	1	Português	Prof. Bruno	Sala Normal 1
2	7ºA	Seg	4	1	Inglês	Prof. Diana	Sala Normal 1
3	7ºA	Ter	1	1	Português	Prof. Bruno	Sala Normal 2
4	7ºA	Ter	2	1	Matemática	Prof. Ana	Sala Normal 2
5	7ºA	Ter	4	2	Educação Física	Prof. Eduardo	Ginásio

	turma	dia	periodo	duracao	disciplina	professor	sala
6	7ºA	Qua	1	1	Ciências	Prof. Carla	Laboratório
7	7ºA	Qua	2	1	Inglês	Prof. Diana	Sala Normal 1
8	7ºA	Qua	3	1	Português	Prof. Bruno	Sala Normal 1
9	7ºA	Qua	4	1	Matemática	Prof. Ana	Sala Normal 2
10	7ºA	Qui	2	1	Português	Prof. Bruno	Sala Normal 1
11	7ºA	Qui	4	1	Matemática	Prof. Ana	Sala Normal 1
12	7ºA	Qui	5	1	Ciências	Prof. Carla	Laboratório
13	7ºA	Sex	2	1	Ciências	Prof. Carla	Laboratório
14	7ºA	Sex	3	1	Matemática	Prof. Ana	Sala Normal 1
15	7ºA	Sex	4	1	História	Prof. Diana	Sala Normal 1
16	7ºB	Seg	1	1	Ciências	Prof. Carla	Laboratório
17	7ºB	Seg	2	1	Matemática	Prof. Ana	Sala Normal 1
18	7ºB	Seg	3	1	Inglês	Prof. Diana	Sala Normal 6
19	7ºB	Seg	4	2	Educação Física	Prof. Eduardo	Ginásio
20	7ºB	Ter	1	1	Matemática	Prof. Ana	Sala Normal 1
21	7ºB	Ter	2	1	Português	Prof. Bruno	Sala Normal 1
22	7ºB	Ter	3	1	Ciências	Prof. Carla	Laboratório
23	7ºB	Ter	5	1	História	Prof. Diana	Sala Normal 1
24	7ºB	Qua	4	1	Português	Prof. Bruno	Sala Normal 1
25	7ºB	Qui	1	1	Português	Prof. Bruno	Sala Normal 1
26	7ºB	Qui	2	1	História	Prof. Diana	Sala Normal 6
27	7ºB	Qui	3	1	Matemática	Prof. Ana	Sala Normal 1
28	7ºB	Qui	4	1	Ciências	Prof. Carla	Laboratório
29	7ºB	Sex	1	1	Português	Prof. Bruno	Sala Normal 1
30	7ºB	Sex	2	1	Matemática	Prof. Ana	Sala Normal 1
31	7ºB	Sex	3	1	Inglês	Prof. Diana	Sala Normal 6

H1 incremental

	turma	dia	periodo	duracao	disciplina	professor	sala
0	7ºA	Seg	1	1	Português	Prof. Bruno	Sala Normal 1
1	7ºA	Seg	2	1	Ciências	Prof. Carla	Laboratório
2	7ºA	Seg	3	1	Matemática	Prof. Ana	Sala Normal 1

	turma	dia	periodo	duracao	disciplina	professor	sala
3	7ºA	Seg	4	2	Educação Física	Prof. Eduardo	Ginásio
4	7ºA	Ter	1	1	Português	Prof. Bruno	Sala Normal 1
5	7ºA	Ter	2	1	Ciências	Prof. Carla	Laboratório
6	7ºA	Ter	3	1	Matemática	Prof. Ana	Sala Normal 1
7	7ºA	Ter	4	1	Inglês	Prof. Diana	Sala Normal 1
8	7ºA	Qua	3	1	Português	Prof. Bruno	Sala Normal 1
9	7ºA	Qua	4	1	Ciências	Prof. Carla	Laboratório
10	7ºA	Qua	5	1	Matemática	Prof. Ana	Sala Normal 1
11	7ºA	Qui	3	1	Inglês	Prof. Diana	Sala Normal 1
12	7ºA	Qui	4	1	História	Prof. Diana	Sala Normal 2
13	7ºA	Qui	5	1	Português	Prof. Bruno	Sala Normal 1
14	7ºA	Sex	3	1	Matemática	Prof. Ana	Sala Normal 1
15	7ºA	Sex	4	1	História	Prof. Diana	Sala Normal 2
16	7ºB	Seg	3	1	Inglês	Prof. Diana	Sala Normal 6
17	7ºB	Seg	4	1	Matemática	Prof. Ana	Sala Normal 1
18	7ºB	Ter	1	1	Ciências	Prof. Carla	Laboratório
19	7ºB	Ter	2	1	Português	Prof. Bruno	Sala Normal 1
20	7ºB	Ter	3	1	História	Prof. Diana	Sala Normal 6
21	7ºB	Ter	4	2	Educação Física	Prof. Eduardo	Ginásio
22	7ºB	Qua	2	1	Português	Prof. Bruno	Sala Normal 1
23	7ºB	Qua	4	1	Matemática	Prof. Ana	Sala Normal 1
24	7ºB	Qui	1	1	Matemática	Prof. Ana	Sala Normal 1
25	7ºB	Qui	2	1	História	Prof. Diana	Sala Normal 1
26	7ºB	Qui	4	1	Português	Prof. Bruno	Sala Normal 1
27	7ºB	Qui	5	1	Ciências	Prof. Carla	Laboratório
28	7ºB	Sex	1	1	Português	Prof. Bruno	Sala Normal 1
29	7ºB	Sex	2	1	Matemática	Prof. Ana	Sala Normal 1
30	7ºB	Sex	3	1	Inglês	Prof. Diana	Sala Normal 6
31	7ºB	Sex	5	1	Ciências	Prof. Carla	Laboratório

H1 resolvido do zero

	turma	dia	periodo	duracao	disciplina	professor	sala
0	7ºA	Seg	3	1	Português	Prof. Bruno	Sala Normal 1
1	7ºA	Seg	4	1	Inglês	Prof. Diana	Sala Normal 1
2	7ºA	Ter	2	1	Matemática	Prof. Ana	Sala Normal 1
3	7ºA	Ter	4	1	Português	Prof. Bruno	Sala Normal 1
4	7ºA	Qua	1	1	Matemática	Prof. Ana	Sala Normal 2
5	7ºA	Qua	2	1	Português	Prof. Bruno	Sala Normal 2
6	7ºA	Qua	3	1	Ciências	Prof. Carla	Laboratório
7	7ºA	Qua	4	1	História	Prof. Diana	Sala Normal 1
8	7ºA	Qui	1	1	Português	Prof. Bruno	Sala Normal 1
9	7ºA	Qui	2	1	Ciências	Prof. Carla	Laboratório
10	7ºA	Qui	3	1	Matemática	Prof. Ana	Sala Normal 1
11	7ºA	Qui	4	2	Educação Física	Prof. Eduardo	Ginásio
12	7ºA	Sex	1	1	Inglês	Prof. Diana	Sala Normal 1
13	7ºA	Sex	2	1	História	Prof. Diana	Sala Normal 1
14	7ºA	Sex	3	1	Matemática	Prof. Ana	Sala Normal 1
15	7ºA	Sex	4	1	Ciências	Prof. Carla	Laboratório
16	7ºB	Seg	1	1	Matemática	Prof. Ana	Sala Normal 1
17	7ºB	Seg	2	1	História	Prof. Diana	Sala Normal 1
18	7ºB	Seg	3	1	Inglês	Prof. Diana	Sala Normal 6
19	7ºB	Seg	4	2	Educação Física	Prof. Eduardo	Ginásio
20	7ºB	Ter	1	1	Matemática	Prof. Ana	Sala Normal 1
21	7ºB	Ter	2	1	Ciências	Prof. Carla	Laboratório
22	7ºB	Ter	5	1	Português	Prof. Bruno	Sala Normal 1
23	7ºB	Qua	1	1	Português	Prof. Bruno	Sala Normal 1
24	7ºB	Qua	2	1	Matemática	Prof. Ana	Sala Normal 1
25	7ºB	Qua	4	1	Ciências	Prof. Carla	Laboratório
26	7ºB	Qua	5	1	História	Prof. Diana	Sala Normal 1
27	7ºB	Qui	2	1	Português	Prof. Bruno	Sala Normal 1
28	7ºB	Qui	4	1	Matemática	Prof. Ana	Sala Normal 1
29	7ºB	Sex	1	1	Português	Prof. Bruno	Sala Normal 6
30	7ºB	Sex	3	1	Inglês	Prof. Diana	Sala Normal 6
31	7ºB	Sex	5	1	Ciências	Prof. Carla	Laboratório

Conclusão

Evidência para R9 (valores desta execução)

- **Alteração de dados_v2/** : a atualização incremental foi 2.1 vezes mais rápida (0.022 s contra 0.045 s) e alterou **2** aula(s), contra **29** na resolução do zero. Os dois horários H1 passaram na validação (0 e 0 erros).
- **Reparação forçada de uma aula de H0:** a atualização incremental foi 1.7 vezes mais rápida (0.027 s contra 0.046 s) e alterou **3** aula(s), contra **26** na resolução do zero. Os dois horários H1 passaram na validação (0 e 0 erros).

A diferença no número de aulas alteradas é a esperada: a resolução do zero não conhece H0 e reorganiza quase todo o horário, enquanto a atualização incremental maximiza as aulas preservadas e só muda as que a alteração obriga a mudar. A diferença de tempo vem de H0 ser dado ao solver como pista: a solução inicial já é quase ótima para o objetivo de preservação, que é também mais simples de provar ótimo do que o dos buracos. Os tempos absolutos variam entre execuções e computadores; o número de aulas alteradas é a medida mais estável.

H0 tem aula(s) da Prof. Ana à sexta nos períodos 4 ou 5. A alteração de dados_v2/ afeta pelo menos uma aula.

O conjunto dados_teste/ acrescenta as turmas 9Z e 8A; mantém as mesmas seis disciplinas.

Os testes automáticos de R1-R8, da atualização H0 → H1 e dos dados adicionais estão no notebook test_horario_escolar.py .

Uso de ferramentas LLM

Usámos o **ChatGPT** como apoio ao longo do trabalho. A conversa completa está em: https://chatgpt.com/s/cx_6ac29be418648191b9d999922da25f31. Os registos dessa conversa estão transcritos no apêndice abaixo (e no ficheiro apendice_conversas.md).

No código, cada função em que o ChatGPT contribuiu tem um comentário # LLM (ChatGPT) – Registo N: ... logo a seguir à documentação, que indica o registo da conversa e o que foi sugerido. As funções sem esse comentário não têm registo na conversa.

Função no código	Registo	Contributo do ChatGPT
carregar_dados	1	Reunir a leitura dos quatro CSV numa função

Função no código	Registro	Contributo do ChatGPT
<code>preparar_dados</code>	1, 2	Conversões de tipos e validações dos dados
<code>criar_modelo</code>	4	Uma variável booleana por colocação possível de aula
<code>ocupantes</code>	6	Uma aula dupla ocupa o período de início e o seguinte
<code>adicionar_restricoes</code>	5, 6	Formulação de R1-R7 como somas de variáveis
<code>adicionar_objetivo_buracos</code>	7	Lógica de detecção de buracos nos períodos 2 a 4
<code>resolver_com_salas</code>	8	Função de resolução (<code>resolver_modelo</code>)
<code>construir_horario</code>	8	Extração das aulas escolhidas (<code>extrair_horario</code>)
<code>apresentar_horario</code>	1, 8	Tabela ordenada por turma, dia e período
<code>validar_horario</code>	1, 8	Validador independente do solver
<code>contar_alteracoes</code>	1, 8	Emparelhar ocorrências da mesma turma e disciplina
<code>atualizar_horario</code>	1, 8	Reaproveitar H0 e preservar as aulas válidas
<code>gerar_horario</code>	9	Fluxo completo dados → H0
<code>comparar_metodos</code>	1, 9	Comparar com a resolução do zero (tempo e aulas alteradas)
<code>test_horario_escolar.py</code>	—	Testes gerados a partir do prompt indicado nesse notebook

Diferenças entre a conversa e o código final

- `criar_tempos` (Registro 3) foi substituída pelas constantes `DIAS` e `PERIODOS`, definidas no início do notebook.
- As quatro funções de restrições sugeridas (`adicionar_restricoes_turmas`, `_professores`, `_salas` e `_diarias`, Registos 1, 5 e 6) foram juntadas em `adicionar_restricoes`.
- `resolver_modelo` e `extrair_horario` (Registro 8) passaram a chamar-se `resolver_com_salas` e `construir_horario`. A atribuição de salas concretas (`atribuir_salas_normais`) foi acrescentada depois.

Apêndice — registos de orientação do projeto

Este ficheiro guarda as transcrições de apoio usadas durante a elaboração. A documentação técnica e os resultados finais estão em `horario_escolar.py`.

Registo 1

Declaração de uso de LLMs

Prompt (ChatGPT):

Explica-me os passos que precisamos para chegar ao produto final, e explica o que cada função que vai ser desenvolvida tem de fazer.

Resposta do LLM:

1. Carregar os dados

Já têm `ler_csv`. Essa função abre um CSV, verifica se contém as colunas necessárias e devolve as linhas como uma lista de dicionários.

A seguir, uma função de carregamento pode reunir os quatro ficheiros:

```
def carregar_dados(pasta):
```

```
    ...
```

Deve ler `turmas.csv`, `disciplinas.csv`, `salas.csv` e `disponibilidade_excecoes.csv` da pasta indicada e devolver os dados agrupados. Assim, a mesma função pode carregar tanto `dados/` como `dados_v2/`.

2. Limpar e validar os dados

Os valores lidos de um CSV são texto. Por isso, será preciso preparar os dados para os cálculos e detetar erros antes de tentar gerar um horário.

```
def preparar_dados(dados):
```

```
    ...
```

Esta função pode converter `carga_semanal`, `quantidade` e `periodo` em números; converter `sim / nao` em valores booleanos; e verificar, por exemplo, se os dias e períodos são válidos e se as salas especiais pedidas existem.

3. Definir os tempos possíveis

O trabalho usa cinco dias e cinco períodos por dia. Uma pequena função pode representar cada tempo como um par `(dia, período)`:

```
def criar_tempos():
```

```
    ...
```

A função devolve todas as combinações possíveis, por exemplo, `("Seg", 1)`, `("Seg", 2)` ... até `("Sex", 5)`. Ter essa lista facilita criar as opções de horário para cada aula.

4. Criar o modelo do horário

Com os dados preparados, o programa precisa de criar uma variável de decisão para cada combinação relevante de turma, disciplina e tempo. Essa variável representa uma pergunta: «Esta aula fica neste período?»

```
def criar_modelo(dados):
```

```
    ...
```

Esta função cria o modelo CP-SAT, as variáveis de decisão e as restrições obrigatórias. A separação exata depende da forma como decidirem representar as aulas. O importante é que o modelo consiga expressar as regras do enunciado.

5. Acrescentar as restrições

Para manter o código compreensível, podem criar funções que acrescentam grupos de regras ao modelo:

```
def adicionar_restricoes_turmas(modelo, x, dados):
```

```
    ...
```

Impede que uma turma tenha duas aulas ao mesmo tempo e garante que cada disciplina cumpre a sua carga semanal.

```
def adicionar_restricoes_professores(modelo, x, dados):
```

```
    ...
```

Impede conflitos entre aulas do mesmo professor e bloqueia os períodos em que esse professor está indisponível.

```
def adicionar_restricoes_salas(modelo, x, dados):
```

```
    ...
```

Garante que cada aula usa uma sala compatível e que não são usadas mais salas do mesmo tipo do que as disponíveis.

```
def adicionar_restricoes_diarias(modelo, x, dados):
```

```
    ...
```

Limita a uma ocorrência diária cada disciplina por turma e trata as disciplinas de duplo período como blocos de dois tempos consecutivos no mesmo dia.

Estas funções devem traduzir os requisitos do enunciado para o modelo. A forma concreta dependerá das variáveis que escolherem; por exemplo, representar cada aula individualmente ou representar blocos duplos como uma opção própria.

6. Otimizar e resolver

Depois das restrições, acrescentam o objetivo de reduzir os «buracos» dos professores — tempos livres entre a primeira e a última aula num dia.

```
def adicionar_objetivo_buracos(modelo, x, dados, tempos):
```

```
    ...
```

Depois, uma função executa o solver:

```
def resolver_modelo(modelo, limite_segundos=None):
```

```
    ...
```

Deve correr o CP-SAT, respeitar um eventual limite de tempo e devolver o estado da resolução e os valores das variáveis. O estado é importante: pode haver solução ótima, solução válida ainda não provada ótima, ou nenhuma solução encontrada.

7. Transformar a solução num horário legível

Os valores das variáveis do solver não são ainda uma apresentação amigável. Uma função pode convertê-los numa lista organizada de aulas:

```
def extrair_horario(solver, x, dados):
```

```
    ...
```

Cada aula deve indicar pelo menos a turma, disciplina, professor, dia, período e sala. Depois, outra função pode mostrar o resultado numa tabela:

```
def apresentar_horario(horario):
```

```
    ...
```

No Marimo, podem apresentar uma tabela por turma ou organizar os dias e períodos em grelhas.

8. Verificar o horário

Mesmo que o solver use as restrições, o trabalho pede verificações automáticas para confirmar que o horário final respeita as regras.

```
def validar_horario(horario, dados):
```

```
    ...
```

Deve percorrer o horário produzido e verificar cargas semanais, conflitos de turma e professor, disponibilidade, salas e blocos duplos. Pode devolver uma lista de

erros; se estiver tudo correto, a lista fica vazia.

9. Atualizar o horário quando os dados mudam

Para a parte incremental, será precisa uma função que compare dois horários:

```
def contar_alteracoes(horario_antigo, horario_novo):
```

```
    ...
```

Deve contar quantas aulas mudaram de período ou sala entre H0 e H1. Para comparar corretamente, cada ocorrência de aula precisa de ter uma identificação estável — por exemplo, turma, disciplina e número da ocorrência naquela semana.

A função de atualização incremental pode receber o horário anterior e os novos dados:

```
def atualizar_horario(horario_antigo, dados_novos):
```

```
    ...
```

Deve gerar um novo horário que respeite os dados novos e, ao mesmo tempo, tente preservar as aulas que ainda podem ficar nos mesmos tempos e salas. Depois, comparem o resultado com uma resolução dos mesmos dados começada do zero. Registem o tempo e o número de alterações para apresentar a evidência pedida no enunciado.

Registo 2

Registo de apoio — preparar_dados

Prompt

```
Vamos prosseguir. Dá-me as bases para a construção da função  
preparar_dados .
```

Resposta

A função `preparar_dados` será a etapa entre **ler os CSV** e **construir o modelo do horário**. `ler_csv` devolve valores como texto; `preparar_dados` converte-os para os tipos adequados e deteta dados inválidos. Ainda não cria o horário.

Como `carregar_dados` devolve quatro valores, a nova função pode recebê-los assim:

```
def preparar_dados(turmas, disciplinas, salas, excecoes):
```

```
    ...
```

Registo 3

Registo de apoio — `criar_tempos`

Prompt

Muito bem. Vamos prosseguir para a próxima função do plano elaborado para resolver este TP1.1.

Resposta

A próxima função do plano é `criar_tempos()`. Ela prepara a lista de todos os tempos possíveis da semana, que depois o modelo usará para decidir onde colocar cada aula. O trabalho define **cinco dias e cinco períodos por dia**. A função deve devolver 25 pares `(dia, período)`, desde `("Seg", 1)` até `("Sex", 5)`.

Para construir a lista, percorremos os dias por uma ordem fixa e, para cada dia, percorremos os períodos de 1 a 5. Assim, os tempos ficam sempre organizados da mesma forma.

Registo 4

Registo de apoio — `criar_modelo`

Prompt

Ok, vamos então à próxima.

Resposta

A próxima função do plano é `criar_modelo`. Antes de a escrevermos, precisamos de decidir o que representa uma variável de decisão no modelo CP-SAT.

Uma opção que trata naturalmente as aulas duplas é criar uma variável booleana para cada **possível colocação de uma aula**:

- `False` significa que a aula não começa nesse dia e período.
- `True` significa que começa aí.
- Para uma aula simples, qualquer período pode ser início.
- Para uma aula dupla, só os períodos 1 a 4 podem ser início, porque tem de haver um período seguinte.

Por exemplo, uma variável para Matemática da turma 7.º A, à terça-feira no período 2, representa essa colocação. Pode chamar-se `x_7A_Matematica_Ter_2`.

A função terá este papel:

```
def criar_modelo(dados, tempos):
```

```
...
```

return modelo, x

Deve criar o modelo CP-SAT e as variáveis de decisão. As regras — como impedir conflitos de professores e cumprir as cargas semanais — ficam para as funções de restrições.

Aulas duplas: uma variável ocupa dois períodos, apesar de representar uma única colocação. As restrições terão de considerar o período de início e o período seguinte.

Registo 5

Registo de apoio — restrições das turmas

Prompt

Agora que finalizámos estas funções, vamos começar a tratar das restrições, começando pelas restrições das turmas.

Resposta

As restrições das turmas cobrem duas regras do enunciado:

- **R1:** uma turma não pode ter duas aulas ao mesmo tempo.
- **R2:** cada disciplina tem de cumprir a carga semanal daquela turma.

A regra **R3** (no máximo uma ocorrência diária da mesma disciplina por turma) fica na função `adicionar_restricoes_diarias`.

Representação das aulas

Cada variável booleana representa uma possível colocação de uma aula. Ela fica organizada em quatro níveis:

`x[turma][disciplina][dia][período_de_início]`

Por exemplo, `x["7ºA"]["Matemática"]["Ter"][2]` representa a opção de Matemática do 7.º A começar à terça-feira no período 2. `True` quer dizer que essa colocação foi escolhida; `False`, que não foi escolhida. Uma aula dupla ocupa o período de início e o seguinte.

A função para acrescentar estas regras ao modelo será:

```
def adicionar_restricoes_turmas(modelo, x, dados):
```

```
...
```

Começar por R2: carga semanal

Para cada turma e disciplina, somamos as variáveis de todas as colocações possíveis. Uma aula simples conta como 1 período; uma dupla conta como 2. Essa soma tem de ser igual à `carga_semanal`.

$\text{soma}(\text{colocações escolhidas} \times \text{duração da aula}) = \text{carga semanal}$

Por exemplo, Matemática com carga semanal 4 precisa de quatro aulas simples; Educação Física com carga 2 e duplo período precisa de um bloco duplo.

Registo 6

Registo de apoio — restantes restrições

Prompt (16:10)

Ok, guia-me agora no resto das funções de restrições.

Resposta do ChatGPT

Vamos organizar as restrições por recurso. Usaremos esta convenção para `x`:

`x[turma][disciplina][dia][período_de_início]`

Cada entrada é uma decisão do solver: a aula começa nesse período ou não. Para uma aula dupla, só criamos opções de início nos períodos 1 a 4; se começar no período 3, ocupa os períodos 3 e 4. Isso faz parte da forma de construir `x` e já ajuda a cumprir R4.

Cada função de restrições recebe o mesmo modelo, `x` e os dados preparados. Dentro dela, `modelo.Add(...)` acrescenta condições que o horário tem de respeitar.

Restrições das turmas

A função `adicionar_restricoes_turmas(modelo, x, dados)` trata três regras:

- **R2 — carga semanal:** para cada turma e disciplina, somar as aulas escolhidas. Aulas simples contam 1 período e duplas contam 2. A soma tem de ser igual à carga semanal.
- **R3 — no máximo uma ocorrência diária:** para cada turma, disciplina e dia, a soma das opções escolhidas nesse dia tem de ser no máximo 1.
- **R1 — sem conflitos na turma:** para cada turma, dia e período, somar as aulas que ocupam esse período. O resultado tem de ser no máximo 1. Para uma aula dupla, conta tanto no início como no período seguinte.

R4 fica garantida se `x` só tiver opções de início válidas para aulas duplas e essas aulas ocuparem sempre dois períodos consecutivos.

Restrições dos professores

A função `adicionar_restricoes_professores(modelo, x, dados)` trata:

- **R5 — sem conflitos:** em cada dia e período, somar as aulas que ocupam esse período e pertencem ao mesmo professor. O total tem de ser no máximo 1.
- **R6 — disponibilidade:** se uma aula calhar num período em que o professor não está disponível, essa opção de `x` tem de ser proibida. Para uma aula dupla, verifica-se também o período seguinte.

Restrições das salas

A função `adicionar_restricoes_salas(modelo, x, dados)` trata a capacidade e a compatibilidade das salas:

- Em cada dia e período, conta as aulas que precisam de cada recurso.
- O total não pode ultrapassar a quantidade disponível em `salas.csv`.
- Aulas sem sala especial usam a capacidade normal; aulas com uma sala especial só contam para essa sala, como Laboratório ou Ginásio.

Se quiserem apresentar o nome exato da sala em cada aula, terão de decidir como atribuí-lo depois. Se as salas de um tipo forem equivalentes, podem primeiro garantir a capacidade com estas contagens.

R8 não é uma restrição do solver: cumpre-se carregando os dados dos CSV, como já fizeram.

A ordem de implementação pode ser: carga semanal (R2), limite diário (R3), conflitos das turmas (R1), conflitos e disponibilidade dos professores (R5–R6), e capacidade das salas (R7). Antes de programarmos a primeira função, precisamos de ter `x` construído com essa convenção.

Registo 7

Registo de apoio — objetivo dos buracos

A função deve pedir ao solver para minimizar os tempos livres **entre** a primeira e a última aula de cada professor em cada dia. Os períodos livres antes da primeira aula ou depois da última não contam como buracos.

A lógica é:

1. Para cada professor, dia e período, determinar se há uma aula nesse período. Uma aula dupla ocupa o seu período de início e o seguinte.
2. Considerar como possível buraco cada período interior — períodos 2, 3 e 4.

3. Um período é buraco se o professor tiver pelo menos uma aula antes, pelo menos uma aula depois e estiver livre nesse período.
4. Minimizar a soma desses buracos para todos os professores e dias.

A função terá uma forma semelhante a:

```
def adicionar_objetivo_buracos(modelo, x, dados, tempos):
```

```
    ...
```

Para representar «há aula antes», «há aula depois» e «este período é um buraco», provavelmente vais precisar de algumas variáveis auxiliares booleanas. Elas não representam aulas novas; servem apenas para o solver calcular o objetivo a partir de `x`.

Depois desta função, a sequência será: executar o solver (`resolver_modelo`), extrair as aulas escolhidas (`extrair_horario`), apresentar o horário, validá-lo e, por fim, implementar a atualização incremental e comparar `H0` com `H1`.

Registo da conversa: 16:38.

Registo 8

Registo de apoio — funções finais

Prompt (16:41)

Prossegue e guia-me no resto das funções.

Resposta do ChatGPT

Claro. Depois de `adicionar_objetivo_buracos`, o trabalho passa das regras do modelo para **resolver, ler e avaliar a solução**. Vamos fazer uma função de cada vez, para poderes implementar e perceber cada etapa.

1. `resolver_modelo(modelo)`

Cria o solver CP-SAT, pede-lhe para resolver o modelo e devolve o resultado. O resultado inclui um estado:

- `OPTIMAL` : encontrou a melhor solução segundo o objetivo;
- `FEASIBLE` : encontrou uma solução válida, mas não provou que seja a melhor;
- `INFEASIBLE` : as restrições não permitem nenhum horário;
- `UNKNOWN` : não conseguiu determinar uma solução.

Só passamos à extração do horário se houver uma solução válida (`OPTIMAL` ou `FEASIBLE`).

2. `extrair_horario(solver, x, dados)`

Percorre a estrutura `x`. Para cada decisão escolhida pelo solver, cria uma entrada legível com a turma, disciplina, professor, dia e período.

Para uma aula dupla, a entrada tem de representar que ocupa o período inicial **e o seguinte**. Também será preciso decidir como atribuir o nome concreto da sala, caso o modelo só controle a capacidade por tipo de sala.

3. `apresentar_horario(horario)`

Recebe a lista criada por `extrair_horario` e mostra-a de forma clara no Marimo. Por exemplo, uma tabela ordenada por turma, dia e período.

4. `validar_horario(horario, dados)`

Verifica o horário gerado independentemente do solver: cargas semanais, conflitos, indisponibilidades, aulas duplas e salas. Idealmente, devolve uma lista de erros; se estiver vazia, o horário passou nas verificações implementadas.

5. `contar_alteracoes(horario_antigo, horario_novo)`

Compara `H0` e `H1` e conta as aulas que mudaram de período ou sala. Para fazer uma comparação coerente, é preciso emparelhar ocorrências da mesma turma e disciplina, incluindo disciplinas com várias aulas por semana.

6. `atualizar_horario(horario_antigo, dados_novos)`

Constrói e resolve um horário com os dados alterados, tentando manter as aulas de `H0` que continuam válidas. Depois, comparamos o tempo de resolução e as aulas alteradas com um `H1` gerado do zero.

O próximo passo concreto é escrever `resolver_modelo`. Para orientar a assinatura pelo teu código, envia-me a função que cria o modelo e acrescenta o objetivo, ou pelo menos diz-me que valores ela devolve.

Registo: a resposta foi indicada como tendo demorado 53 segundos.

Registo 9

Prompt (ChatGPT):

Ok. Agora que está tudo feito, quais são os próximos passos para fechar o projeto?

Resposta do LLM:

Se as funções já estão implementadas, falta **integrá-las, demonstrar que cumprem o enunciado e preparar a entrega**. Sugiro esta ordem:

1. Montar o fluxo completo no notebook

- Carregar e preparar `dados/` .
- Gerar e apresentar `H0` .
- Carregar e preparar `dados_v2/` .
- Gerar e apresentar `H1` aproveitando `H0` .

Link da conversa:

https://chatgpt.com/s/cx_6ac29be418648191b9d9